

ASSIGN5

September 9, 2026

```
[4]: import numpy as np
      y=np.array([1,2])
      z=2*y
      print(z)
```

[2 4]

```
[6]: y=[1,2]
      z=[]
      for n in y:z.append(2*n)
      print(z)
```

[2, 4]

```
[7]: import numpy as np

      arr = np.array([10, 20, 30, 40, 50])

      print("Original array:", arr)

      print("After adding 2:", arr + 2)

      print("After multiplying by 3:", arr * 3)

      print("After dividing by 2:", arr / 2)
```

Original array: [10 20 30 40 50]
After adding 2: [12 22 32 42 52]
After multiplying by 3: [30 60 90 120 150]
After dividing by 2: [5. 10. 15. 20. 25.]

```
[8]: import numpy as np

      arr = np.array([1, 2, 3, 6, 4, 5])

      reverse = arr[::-1]

      print("Original array:", arr)
```

```
print("Reversed array:", reverse)
```

Original array: [1 2 3 6 4 5]

Reversed array: [5 4 6 3 2 1]

```
[9]: import numpy as np

x = np.array([1, 2, 3, 4, 5, 1, 2, 1, 1, 1])

values, counts = np.unique(x, return_counts=True)

most_frequent = values[np.argmax(counts)]

indices = np.where(x == most_frequent)[0]

print("Most frequent value:", most_frequent)
print("Frequency:", counts.max())
print("Indices:", indices)
```

Most frequent value: 1

Frequency: 5

Indices: [0 5 7 8 9]

```
[10]: import numpy as np

y = np.array([1, 1, 1, 2, 3, 4, 2, 4, 3, 3])

values, counts = np.unique(y, return_counts=True)

max_count = counts.max()

most_frequent = values[counts == max_count]

print("Most frequent values:", most_frequent)
print("Frequency:", max_count)

for value in most_frequent:
    indices = np.where(y == value)[0]
    print("Indices of", value, ":", indices)
```

Most frequent values: [1 3]

Frequency: 3

Indices of 1 : [0 1 2]

Indices of 3 : [4 8 9]

```
[11]: import numpy as np
```

```

arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print("1st row, 2nd column:", arr[0, 1])

print("3rd row, 1st column:", arr[2, 0])

```

1st row, 2nd column: 20

3rd row, 1st column: 70

```

[12]: import numpy as np

Ritika = np.linspace(10, 100, 25)

print("Array:")
print(Ritika)

print("Dimensions:", Ritika.ndim)
print("Shape:", Ritika.shape)
print("Total elements:", Ritika.size)
print("Data type:", Ritika.dtype)
print("Total bytes:", Ritika.nbytes)

```

Array:

```

[ 10.    13.75  17.5   21.25  25.    28.75  32.5   36.25  40.    43.75
  47.5   51.25  55.    58.75  62.5   66.25  70.    73.75  77.5   81.25
  85.    88.75  92.5   96.25 100.   ]

```

Dimensions: 1

Shape: (25,)

Total elements: 25

Data type: float64

Total bytes: 200

```

[13]: Ritika_transpose = Ritika.reshape(5, 5)

print("Reshaped array:")
print(Ritika_transpose)

```

Reshaped array:

```

[[ 10.    13.75  17.5   21.25  25.   ]
 [ 28.75  32.5   36.25  40.    43.75]
 [ 47.5   51.25  55.    58.75  62.5 ]
 [ 66.25  70.    73.75  77.5   81.25]
 [ 85.    88.75  92.5   96.25 100.   ]]

```

```
[14]: import numpy as np

      ucs420_Ritika = np.array([
          [10, 20, 30, 40],
          [50, 60, 70, 80],
          [90, 15, 20, 35]
      ])

      print(ucs420_Ritika)
```

```
[[10 20 30 40]
 [50 60 70 80]
 [90 15 20 35]]
```

```
[15]: print("Mean:", np.mean(ucs420_Ritika))
```

```
Mean: 43.333333333333336
```

```
[16]: print("Median:", np.median(ucs420_Ritika))
```

```
Median: 37.5
```

```
[17]: print("Maximum:", np.max(ucs420_Ritika))
```

```
Maximum: 90
```

```
[18]: print("Unique elements:", np.unique(ucs420_Ritika))
```

```
Unique elements: [10 15 20 30 35 40 50 60 70 80 90]
```

```
[19]: reshaped_ucs420_Ritika = ucs420_Ritika.reshape(4, 3)

      print("Reshaped array:")
      print(reshaped_ucs420_Ritika)
```

```
Reshaped array:
[[10 20 30]
 [40 50 60]
 [70 80 90]
 [15 20 35]]
```

```
[ ]:
```